GATE
CS & DA

Artificial Intelligence

Informed Search

Lecture No. 04

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic — A*, WA*
- Topic — AO*
- Topic
- Topic
- Topic

Topics to be Covered

- Topic — Question Concept No of open & close node
- Topic — AO*
- Topic — IDA*
- Topic
- Topic

The concept of open and closed nodes list ...

Open list

Close list

(GBFS)

I → ω

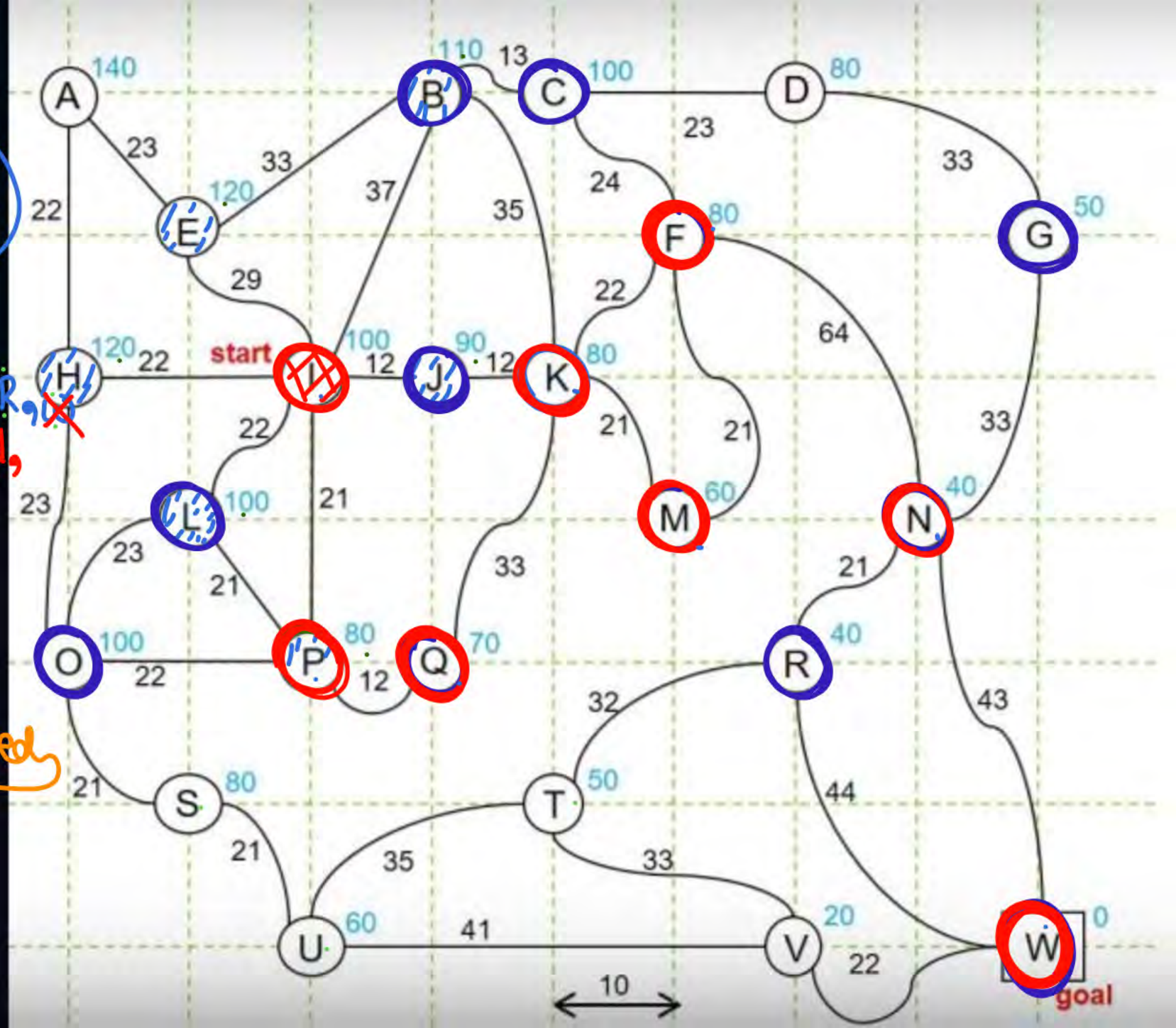
Nodes generated 17

Openlist: ~~I~~, H, E, J, B, ~~R~~, L, O, ~~Q~~, ~~A~~, ~~N~~, ~~M~~, ~~F~~, C, ~~D~~, G, R, ~~ω~~

Close list: I, P, Q, K, M, F, N, ω.

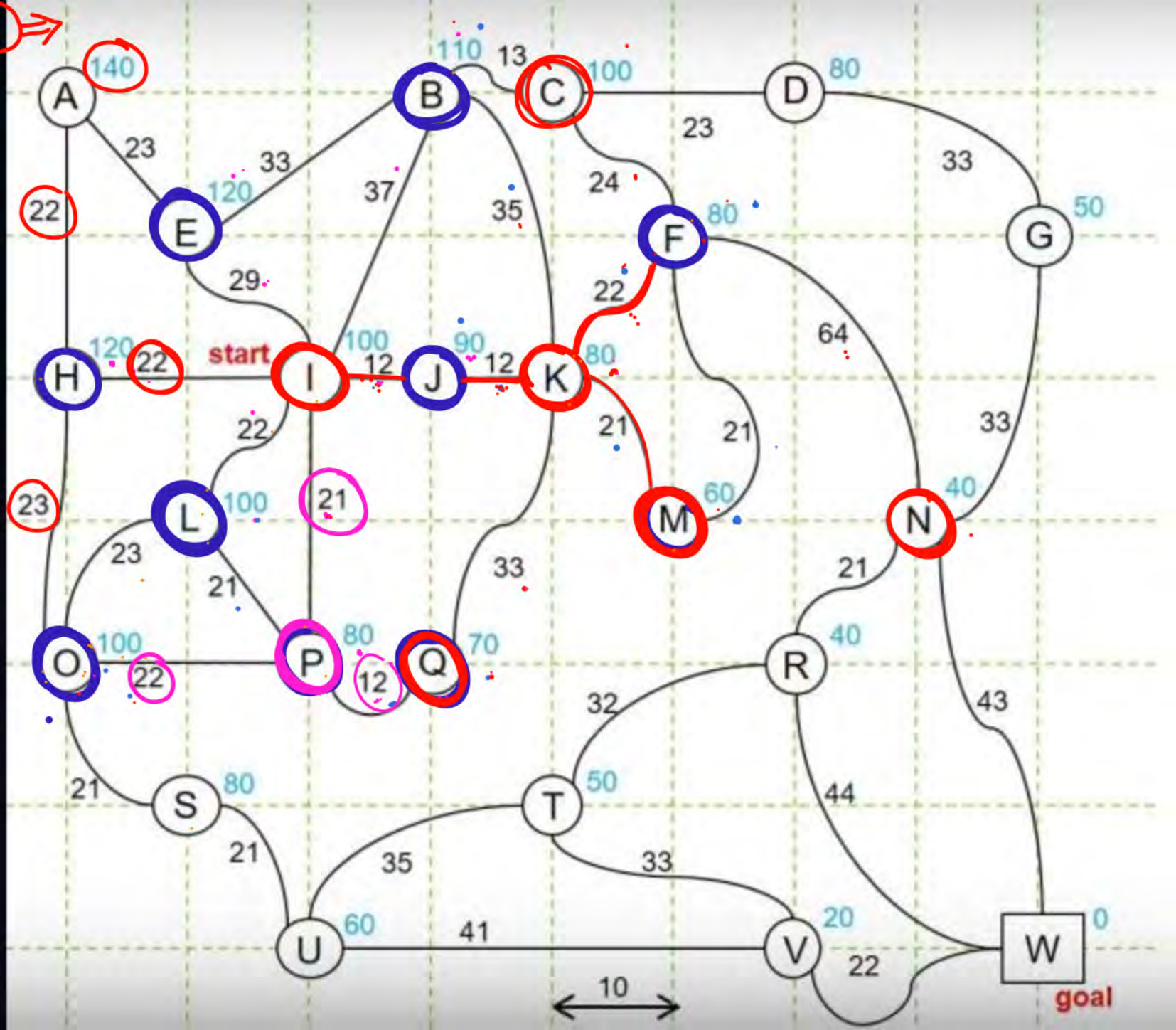
⇒ (Close list ⇒ 8 Nodes)

8 Node Inspected



	I	P	J	Q	K	M	L	F
A								
B								
C	140							
D								
E	149	149	149	149	149	149		
F					126	126		
G								
H	142	142	142	142	142			
I	0	0	0	0	0	0		
J	102	102	102	102	102	102		
K			104	104	104	104		
L	122	122	122	122	122	122		
M					105	105		
N	150							
O		143	143	143	143	143		
P	101	101	101	101	101	101		
Q		103	103	103	103	103		
R								
S								
T								
U								
V								
W								

open list

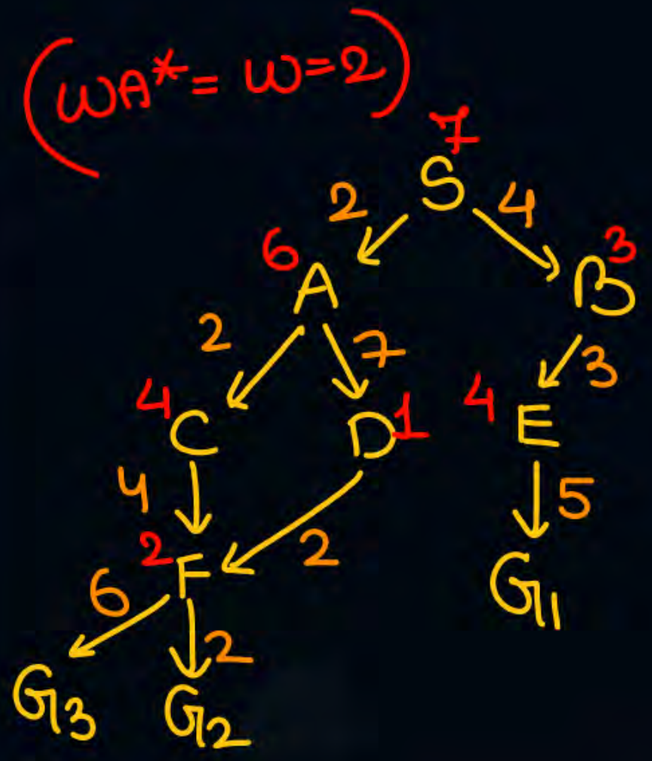


Consider a graph with the following nodes and edges:

- **Nodes:** $S, A, B, C, D, E, F, G1, G2, G3$
- **Edges & Costs:**
  - $S \to A$: 2
  - $S \to B$: 4
  - $A \to C$: 2
  - $A \to D$: 7
  - $B \to E$: 3
  - $C \to F$: 4
  - $D \to F$: 2
  - $E \to G1$: 5
  - $F \to G2$: 2
  - $F \to G3$: 6

Heuristic Estimates (h):

- $h(S) = 7$
- $h(A) = 6$
- $h(B) = 3$
- $h(C) = 4$
- $h(D) = 1$
- $h(E) = 4$
- $h(F) = 2$
- $h(G1) = 0$
- $h(G2) = 0$
- $h(G3) = 0$



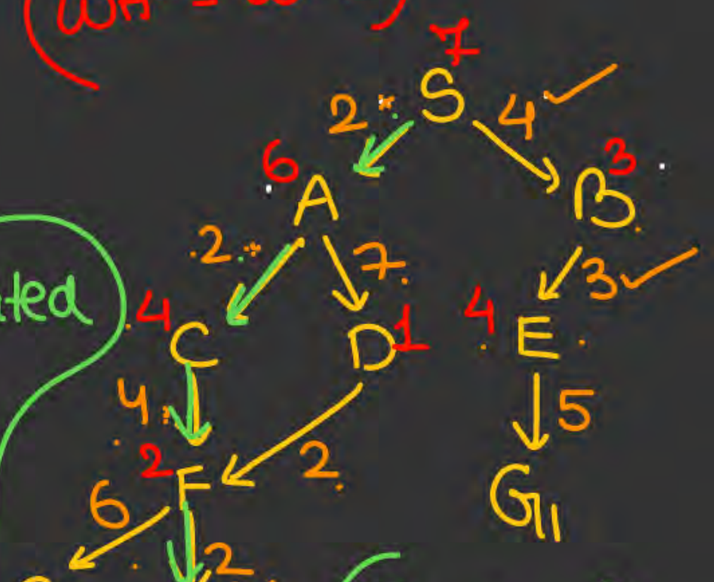
Close list

$f = w + 2 \times h$

$(WA^* = w = 2)$

		S →	B	A	D	C	F	G_2
S	0	0	0	0	0	0	0	
A	14	14	14	14	14	14	14	
B	10	10	10	10	10	10	10	
C	∞	∞	12	12	12	12	12	
D	∞	∞	11	11	11	11	11	
E	∞	15	15	15	15	15	15	
F	∞	∞	∞	15	12	12	12	
G_1	∞	∞	∞	∞	∞	∞	∞	
G_2	∞	∞	∞	∞	∞	∞	10	
G_3	∞	∞	∞	∞	∞	∞	14	

- Node generated Open 9
- Node inspected 7



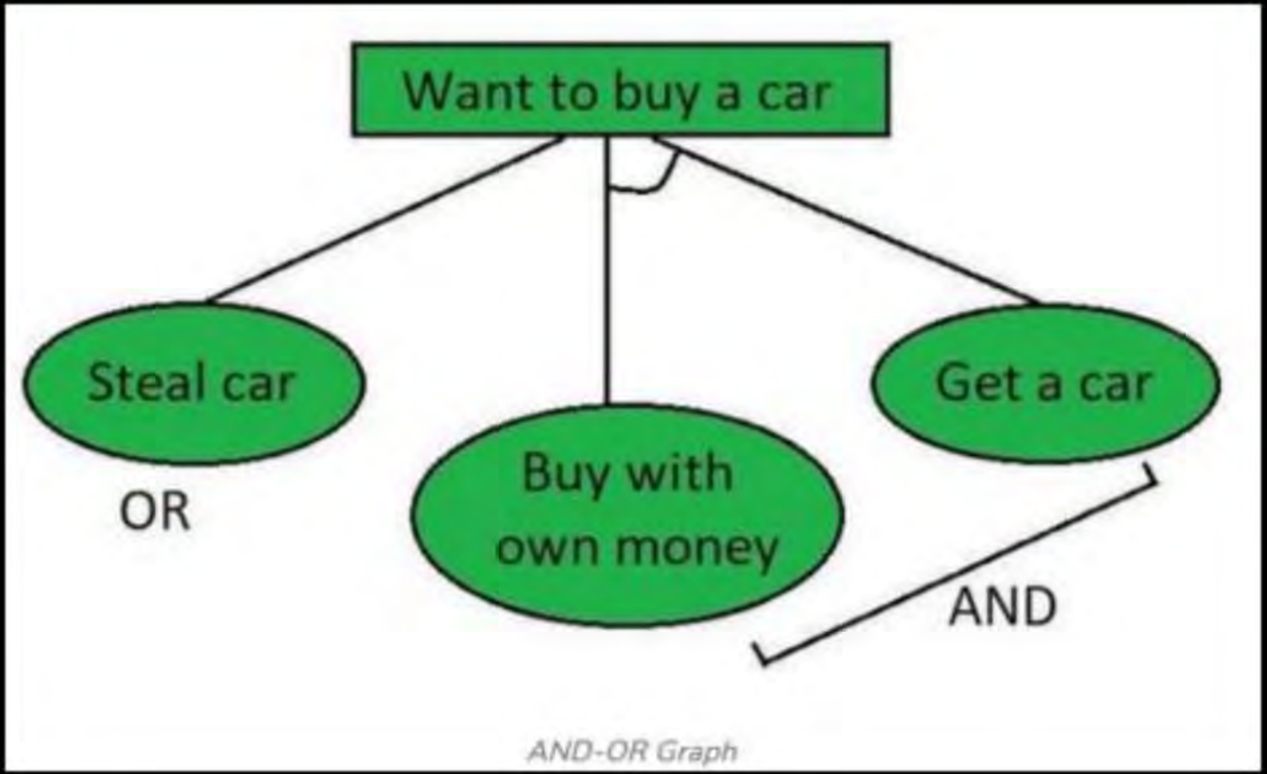
Goal G_2

10

AO* algorithm – Artificial intelligence

- The AO* method divides any given difficult problem into a smaller group of problems that are then resolved using the AND-OR graph concept.
- AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.

AO* algorithm – Artificial intelligence

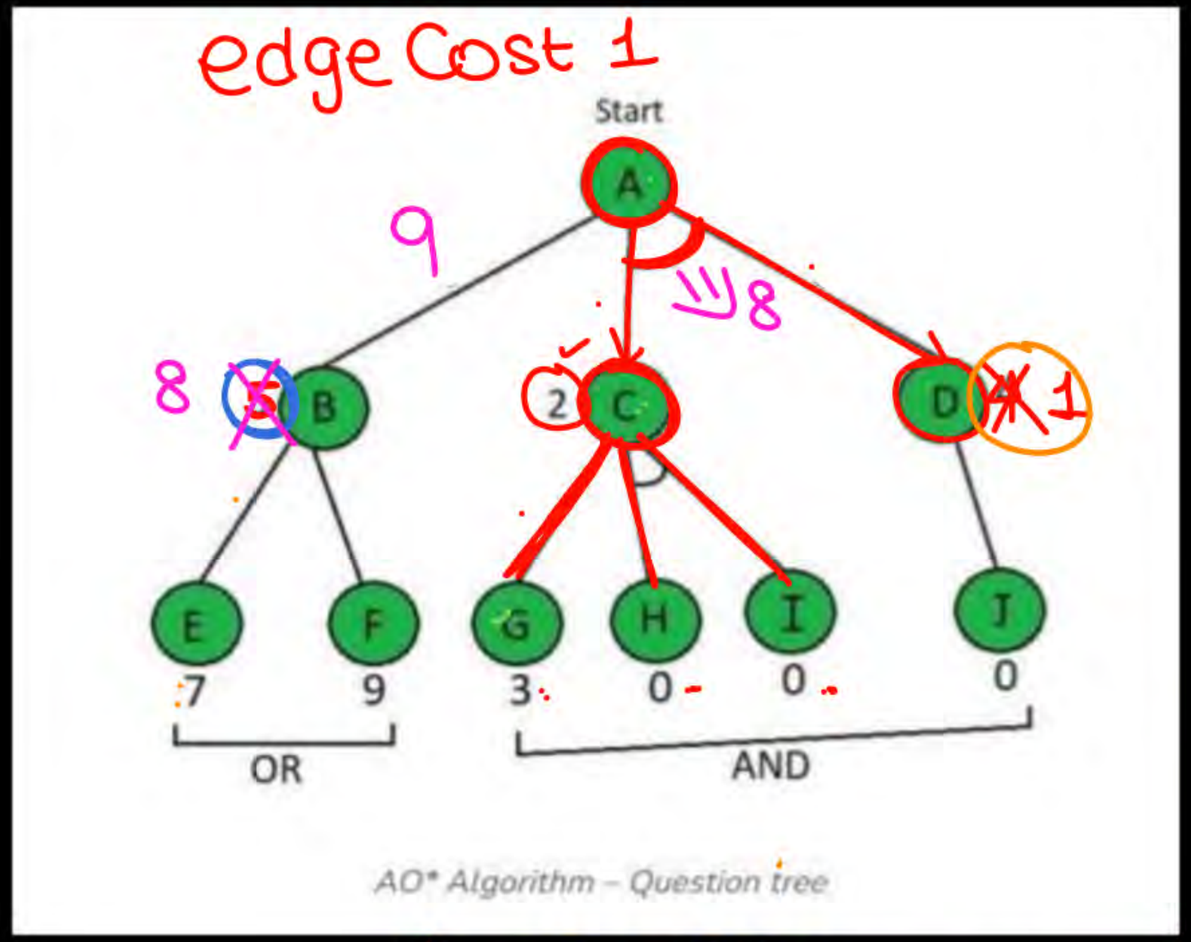


AO* algorithm – Artificial intelligence

- Here we have the concept of forward and backward propagation

AO* algorithm – Artificial intelligence

- In the above figure, the buying of a car may be broken down into smaller problems or tasks that can be accomplished to achieve the main goal in the above figure, which is an example of a simple AND-OR graph. The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the AO* algorithm, and the best path is identified by heuristics. The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.



Forward

$A \to B$ OR $A \to C \$ D$

$\cancel{5} + 1 = 8 + 1 = 9$

$A \to C + A \to D$

$2+1 + \cancel{4}\,1 + 1 \Rightarrow 8$

$2+1+1+1 \Rightarrow 5$

Select

$A \to B \Rightarrow$ Now @ B

$B \to E \to 1+7 = 8$

OR

$B \to F \to 9+1 = 10$

Back Propagation

Heuristic update

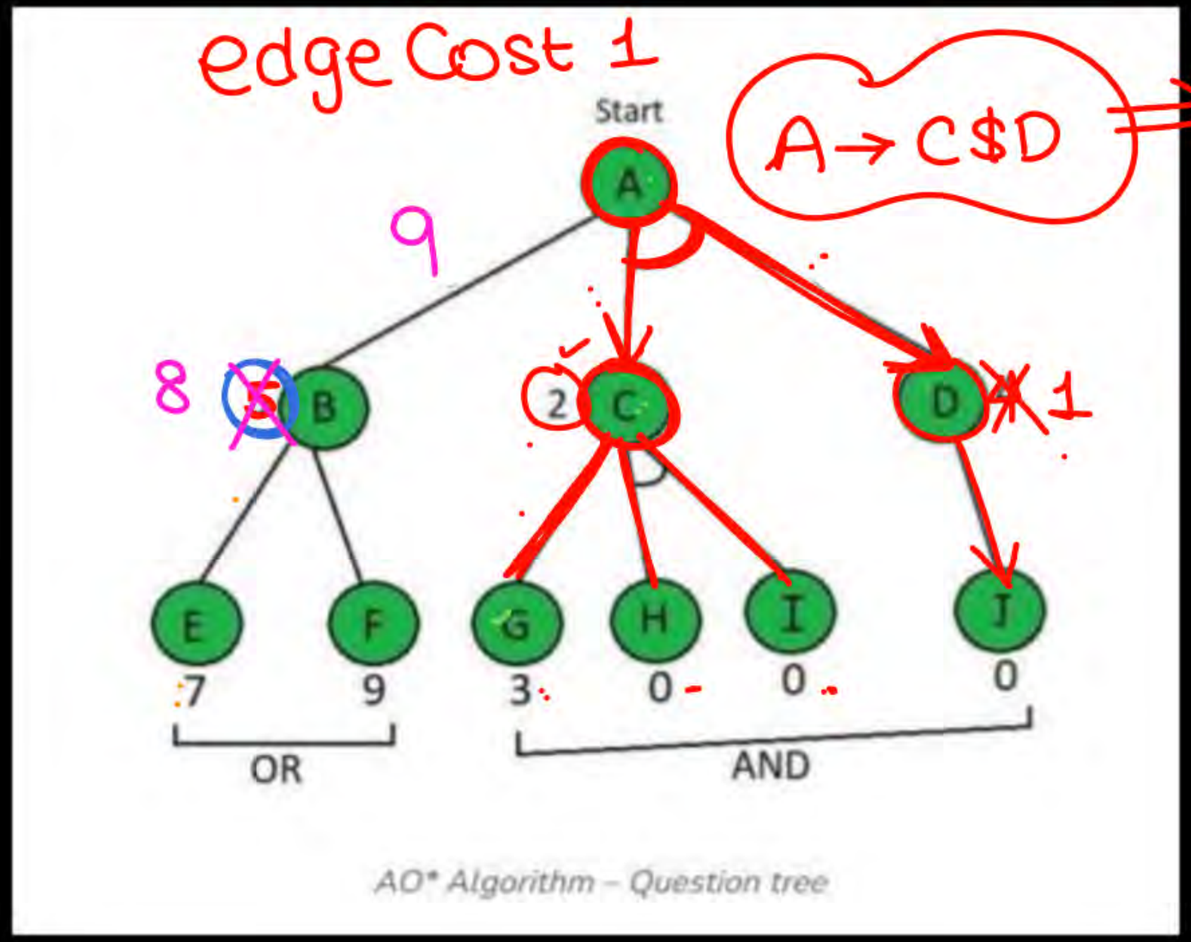
Now $A \to C \$ D$

$C \to G \Rightarrow 4$

OR

$C \to H \$ I \Rightarrow 2$

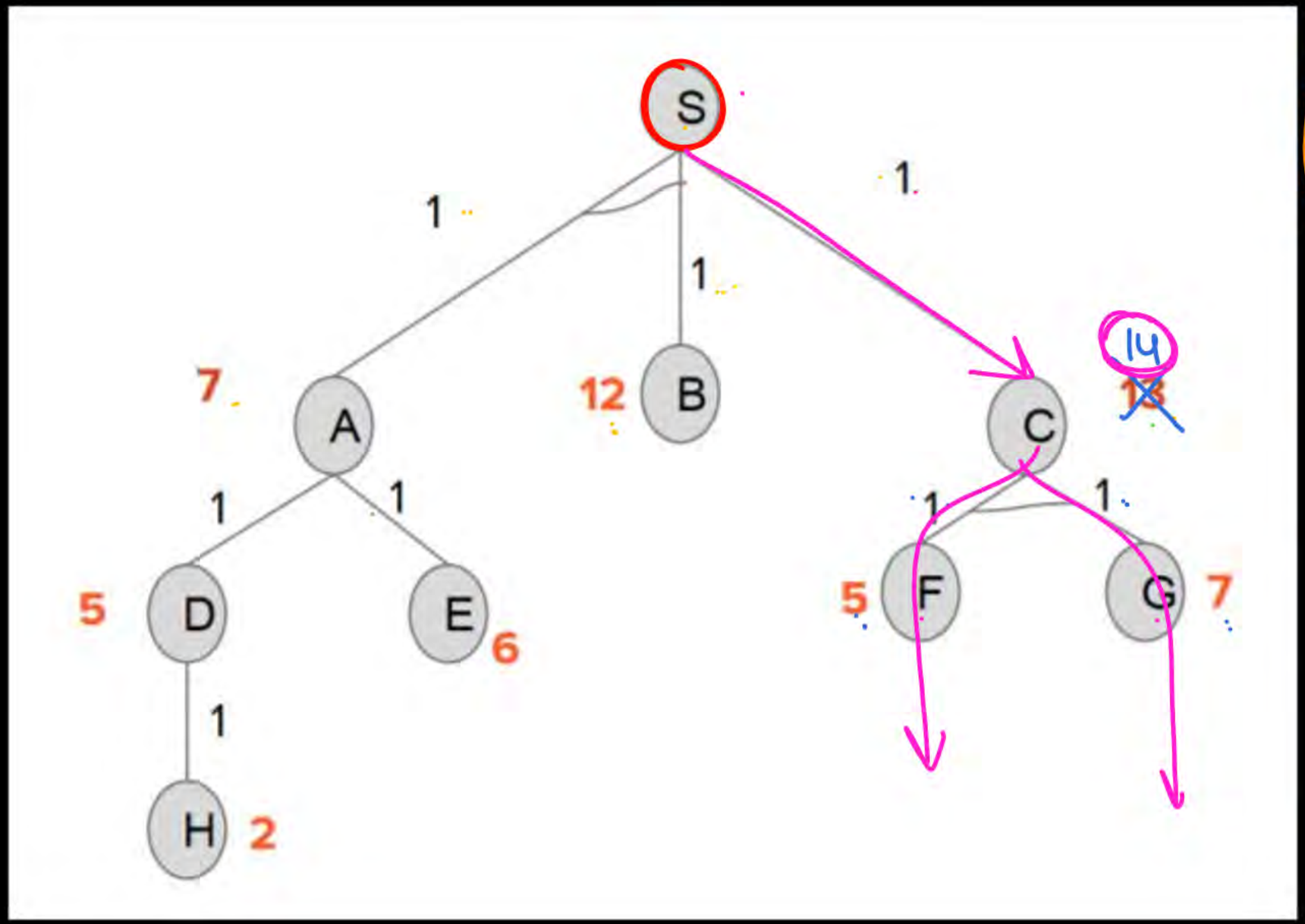
$D \to J \Rightarrow 1$



A → C$D ⇒ 5

A → D → J

A → C$D → J



$S \to C \Rightarrow \cancel{13}\,14 + 1 = \cancel{14}\,15$

OR

$S \to A\$B \Rightarrow 21$

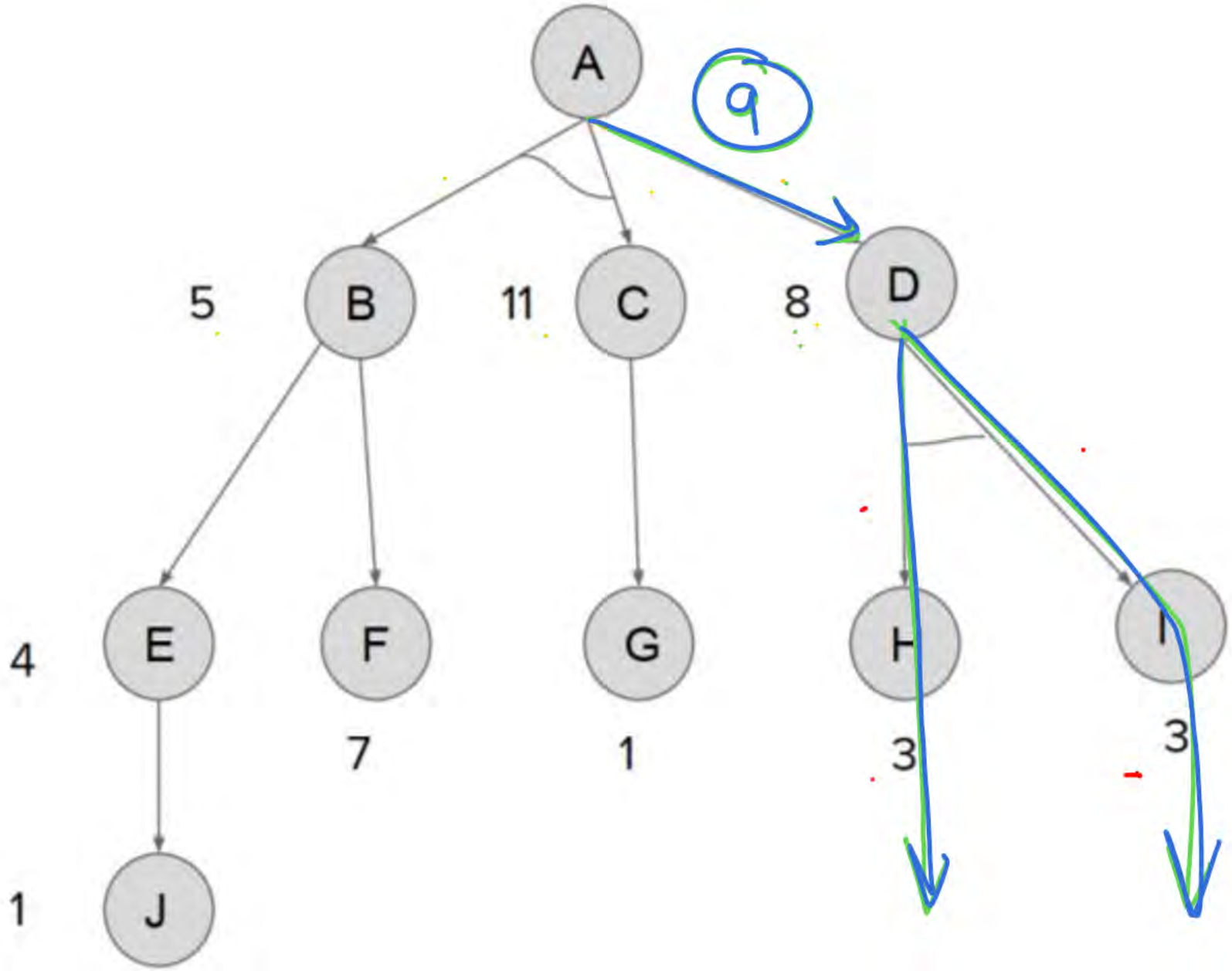
Choose

$C \to F\$G$

14 ⇐ { $C \to F \to$ goal and $C \to G \to$ goal

$S \to C$ { $\to F \to$ goal and $\to G \to$ goal

15



A → D ⇒ 8+1=9

OR

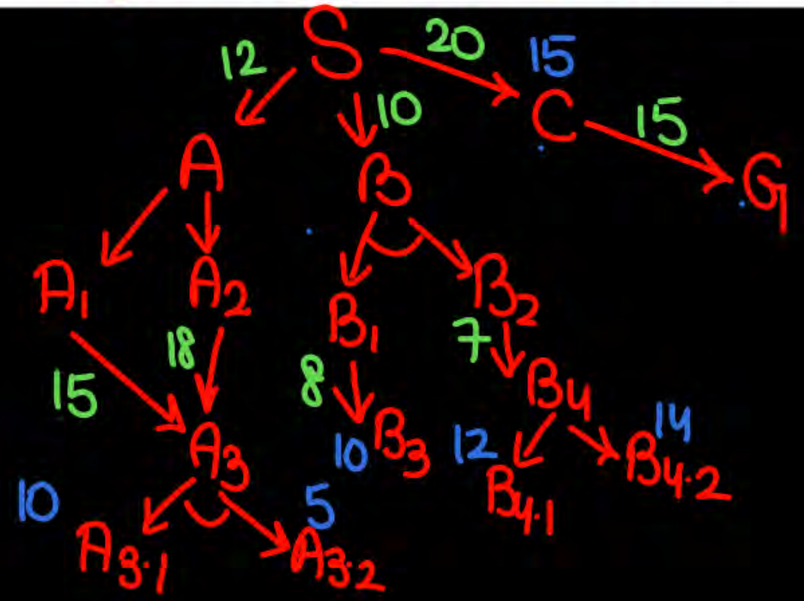
A → C$B = 5+1+1+1

⇒ 18

D → H$I

3+3+2=8

- **S** (Start) can lead to **A**, **B**, or **C**.
  - **A** is an OR node with two alternative paths: **A1** or **A2**.
  - **B** is an AND node that requires the completion of both **B1** and **B2**.
  - **C** directly leads to the goal **G**.
- **A1** and **A2** both lead to **A3**, which is an AND node that must complete two sub-tasks **A3.1** and **A3.2** before reaching **G**.
- **B1** leads to **B3** directly, and **B2** leads to **B4**, which is an OR node with two alternatives: **B4.1** or **B4.2**. anyPath Cost not given=0.



Costs:

- **S → A**: 12 units
- **S → B**: 10 units
- **S → C**: 20 units
- **A1 → A3**: 15 units
- **A2 → A3**: 18 units
- **A3.1 → G**: 10 units
- **A3.2 → G**: 5 units
- **B1 → B3**: 8 units
  B3 → G=10
- **B2 → B4**: 7 units
- **B4.1 → G**: 12 units
- **B4.2 → G**: 14 units
- **C → G**: 15 units

S → A → goal 12+30 ⇒ 42

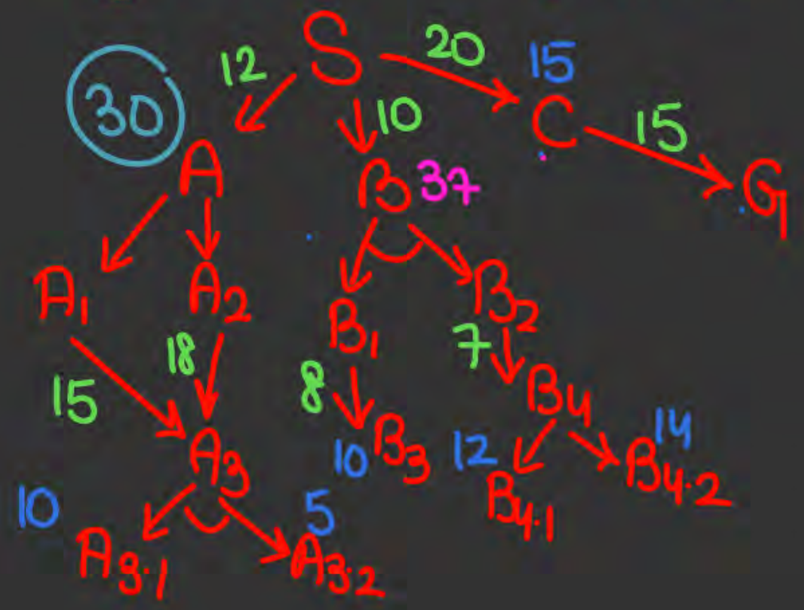
OR

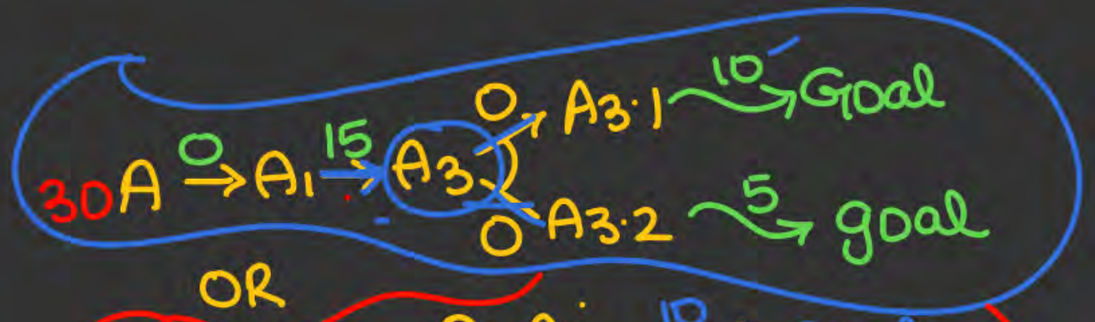
S → B ⇒ goal 10+37 ⇒ 47

OR

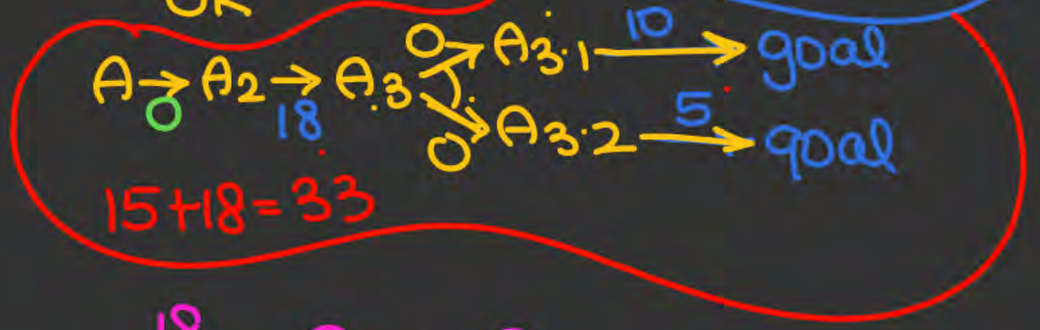
S → C → goal 20+15 ⇒ 35

anyPath cost not given = 0.

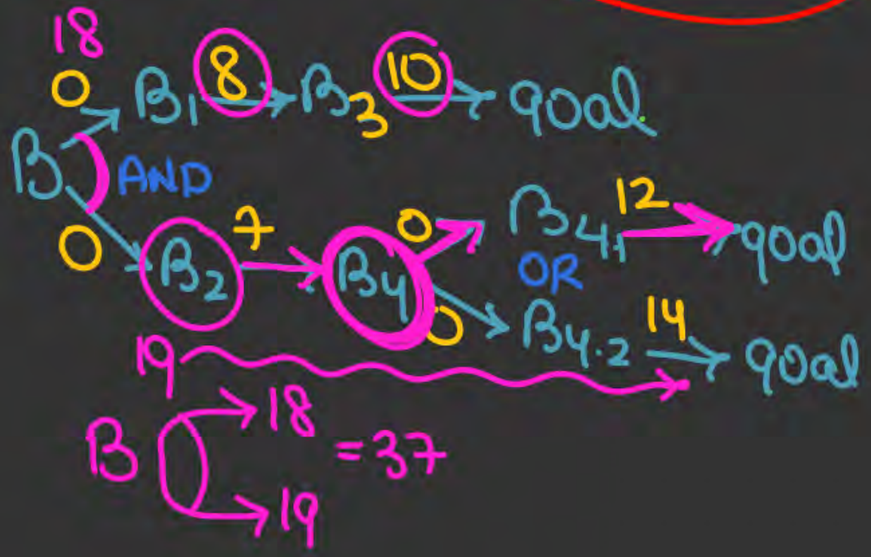




OR



15+18=33



The costs for expanding nodes are as follows:

- Node `B` leads to nodes `E` and `F` with costs 3 and 2, respectively.
- Node `C` leads to node `G` with a cost of 4.
- Node `D` leads to nodes `H` and `F` with costs 3 and 5, respectively.

The goal is to find the minimum cost path from the start node `A` to the goal node, where the goal nodes are `E`, `F`, `G`, and `H`.

AO* algorithm – Artificial intelligence

- In the above figure, the buying of a car may be broken down into smaller problems or tasks that can be accomplished to achieve the main goal in the above figure, which is an example of a simple AND-OR graph. The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the AO* algorithm, and the best path is identified by heuristics. The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.

$A^* \Rightarrow$
$AO^* \Rightarrow$

Why AO^* not optimal

$\Downarrow$

$\Rightarrow$ This algorithm donot analyze the whole tree, thus cannot give optimal Result.

AO* vs A*

1. Purpose and Problem Structure

- *AO Algorithm:**
  - **Used for**: Solving problems represented as AND-OR graphs.
  - **Problem Structure**: Suitable for problems where the solution can be a combination of different strategies or sub-solutions (AND-OR graphs).
- *A Algorithm:**
  - **Used for**: Solving single-agent pathfinding problems.
  - **Problem Structure**: Suitable for problems that can be represented as a single sequence of decisions or moves (a simple graph).

AO* vs A*

Optimality

- *AO Algorithm:**
  - **Optimality**: Finds the optimal solution considering both AND and OR conditions, but its optimality depends on the accuracy of the heuristic, if heuristics are Consistent then result will be optimal.
  - **Efficiency**: The algorithm may need to backtrack if a previously chosen path turns out to be non-optimal after further exploration.
- *A Algorithm:**
  - **Optimality**: Guarantees finding the optimal path if the heuristic is admissible (never overestimates the cost).
  - **Efficiency**: A* is considered optimal and complete if the heuristic is both admissible and consistent.

AO* vs A*

. Complexity

- *AO Algorithm:**
  - **Complexity**: More complex due to the need to handle both AND and OR nodes, and the need to maintain multiple paths that might need to be explored simultaneously.
  - **Space and Time**: The complexity depends on the structure of the AND-OR graph and the quality of the heuristic.
- *A Algorithm:**
  - **Complexity**: Generally less complex than AO* as it deals with a simpler graph structure.
  - **Space and Time**: Exponential in the worst case, as A* may explore many nodes, but it is highly efficient with a good heuristic.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- Iterative deepening A* (IDA*) is a graph traversal and path-finding method that can determine the shortest route in a weighted graph between a defined start node and any one of a group of goal nodes.
- It is a kind of iterative deepening depth-first search that adopts the A* search algorithm's idea of using a heuristic function to assess the remaining cost to reach the goal.
- A memory-limited version of A* is called IDA*. It performs all operations that A* does and has optimal features for locating the shortest path, but it occupies less memory.
- Iterative Deepening A Star uses a heuristic to choose which nodes to explore and at which depth to stop, as opposed to Iterative Deepening DFS, which utilizes simple depth to determine when to end the current iteration and continue with a higher depth.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The evaluation function in IDA* looks like this:

$$f(n) = g(n) + h(n)$$
$$f(n) = Actual\ cost + Estimated\ cost$$

- Where h is admissible.
- here,
- f(n) = Total cost evaluation function.
- g(n) = The actual cost from the initial node to the current node.
- h(n) = Heuristic estimated cost from the current node to the goal state.

it is based on the approximation according to the problem characteristics.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

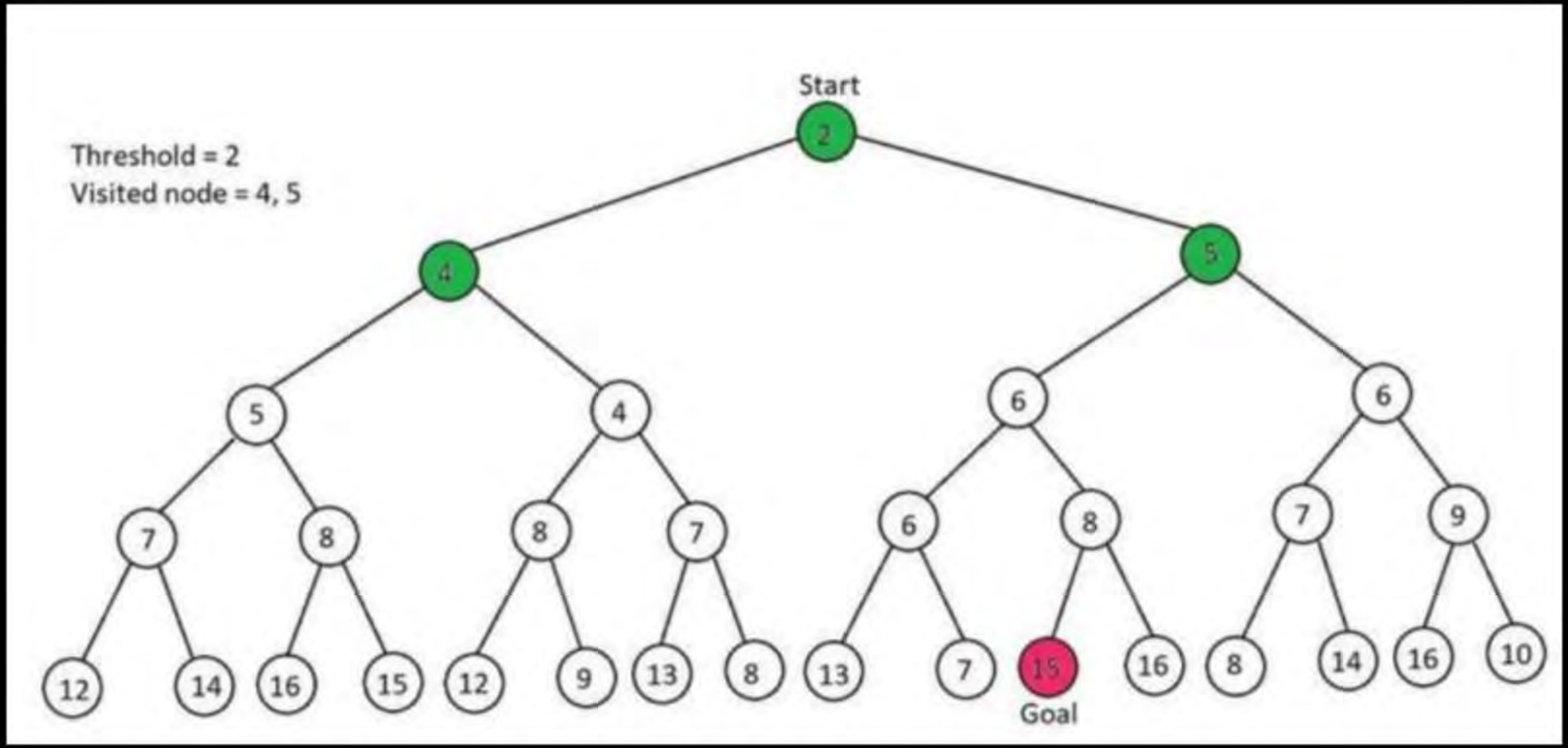
- Step 1: Initialization
- Set the root node as the current node, and find the f-score.
- Sep 2: Set threshold
- Set the cost limit as a threshold for a node i.e the maximum f-score allowed for that node for further explorations.
- Step 3: Node Expansion
- Expand the current node to its children and find f-scores.
- Step 4: Pruning
- If for any node the f-score > threshold, prune that node because it's considered too expensive for that node. and store it in the visited node list.
- Step 5: Return Path
- If the Goal node is found then return the path from the start node Goal node.

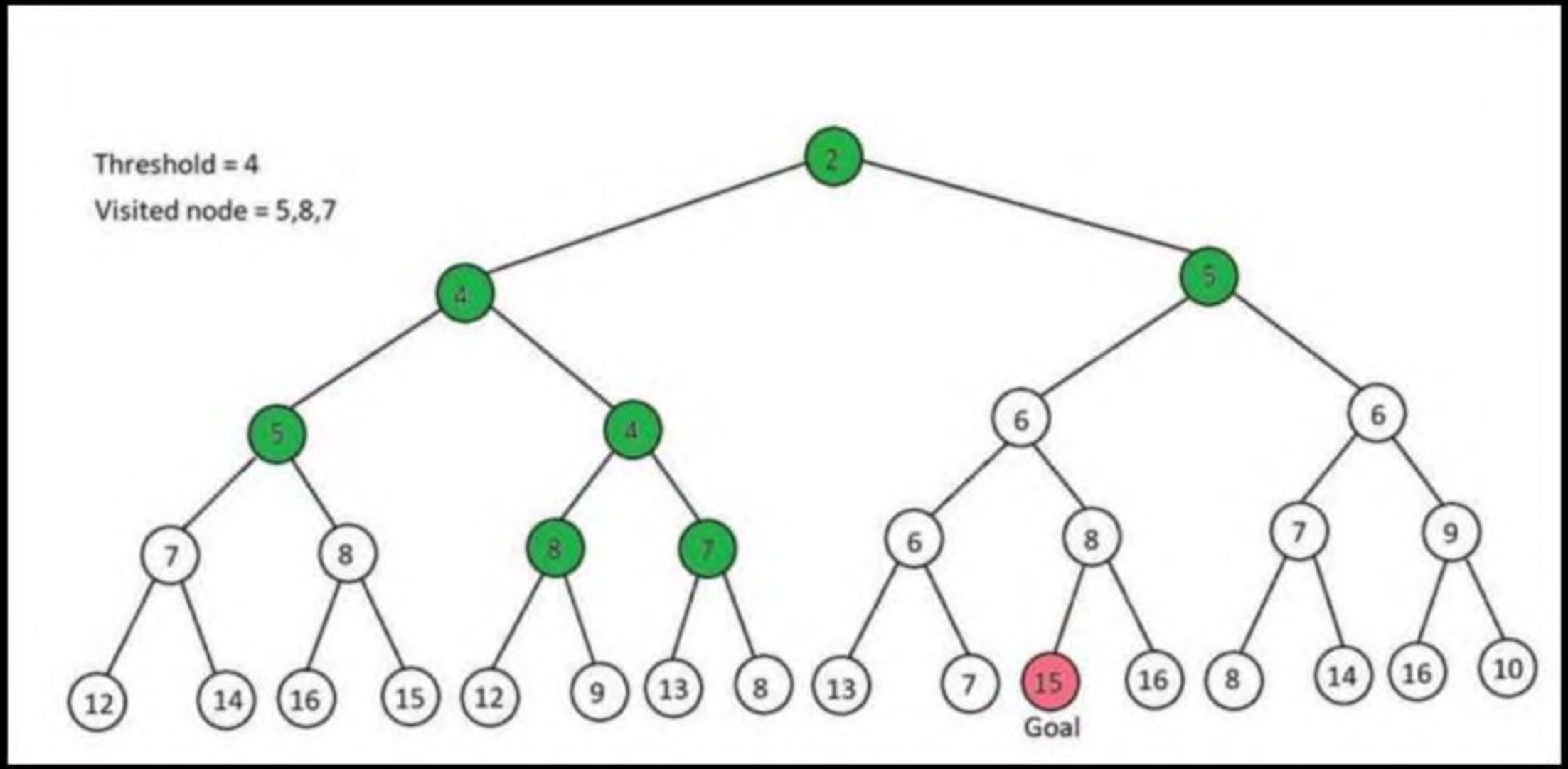
Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

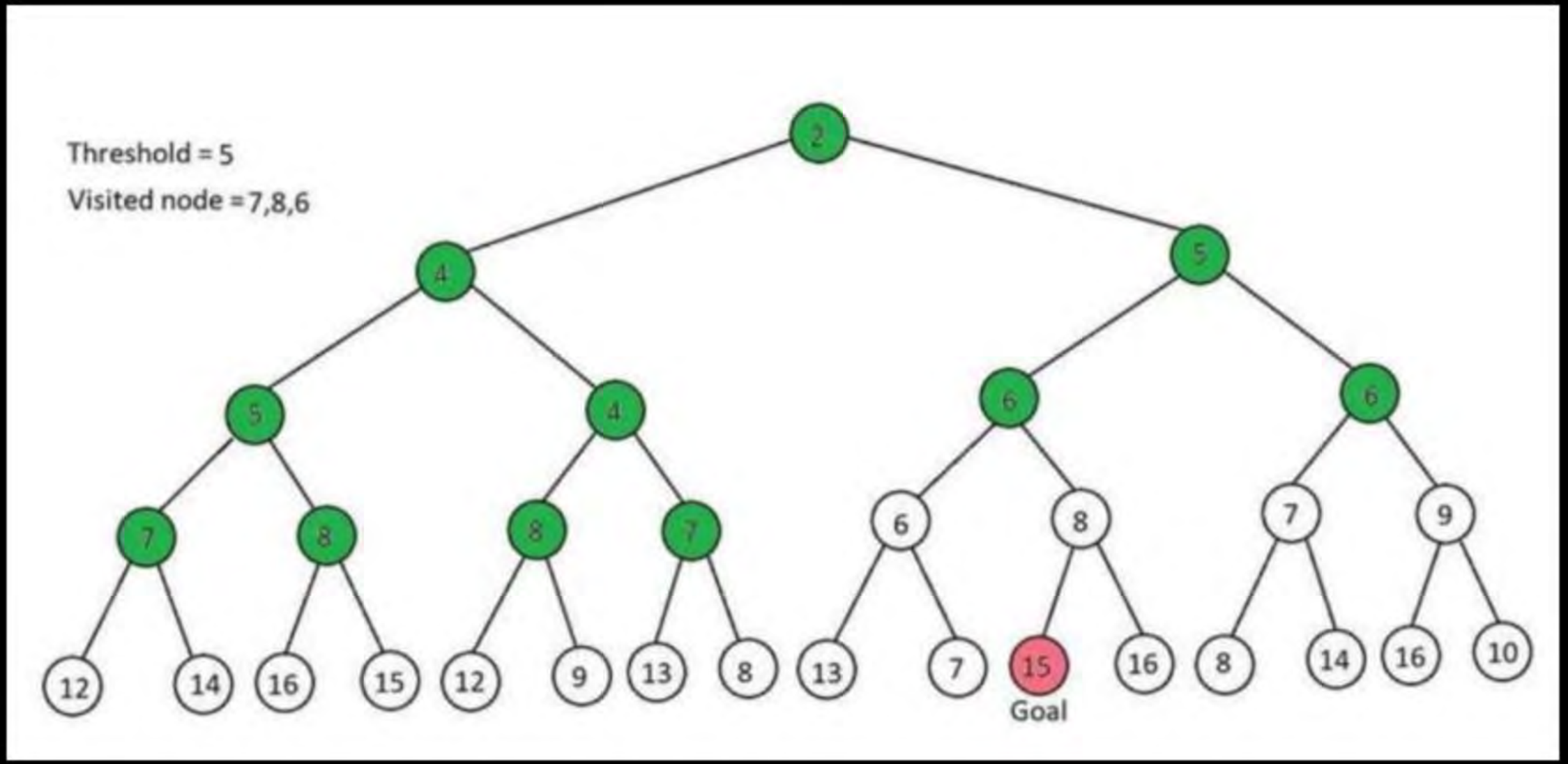
- Step 6: Update the Threshold
- If the Goal node is not found then repeat from step 2 by changing the threshold with the minimum pruned value from the visited node list. And Continue it until you reach the goal node.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Example
- In the below tree, the f score is written inside the nodes means the f score is already computed and the start node is 2 whereas the goal node is 15. the explored node is colored green color.
- so now we have to go to a given goal by using IDA* algorithm.

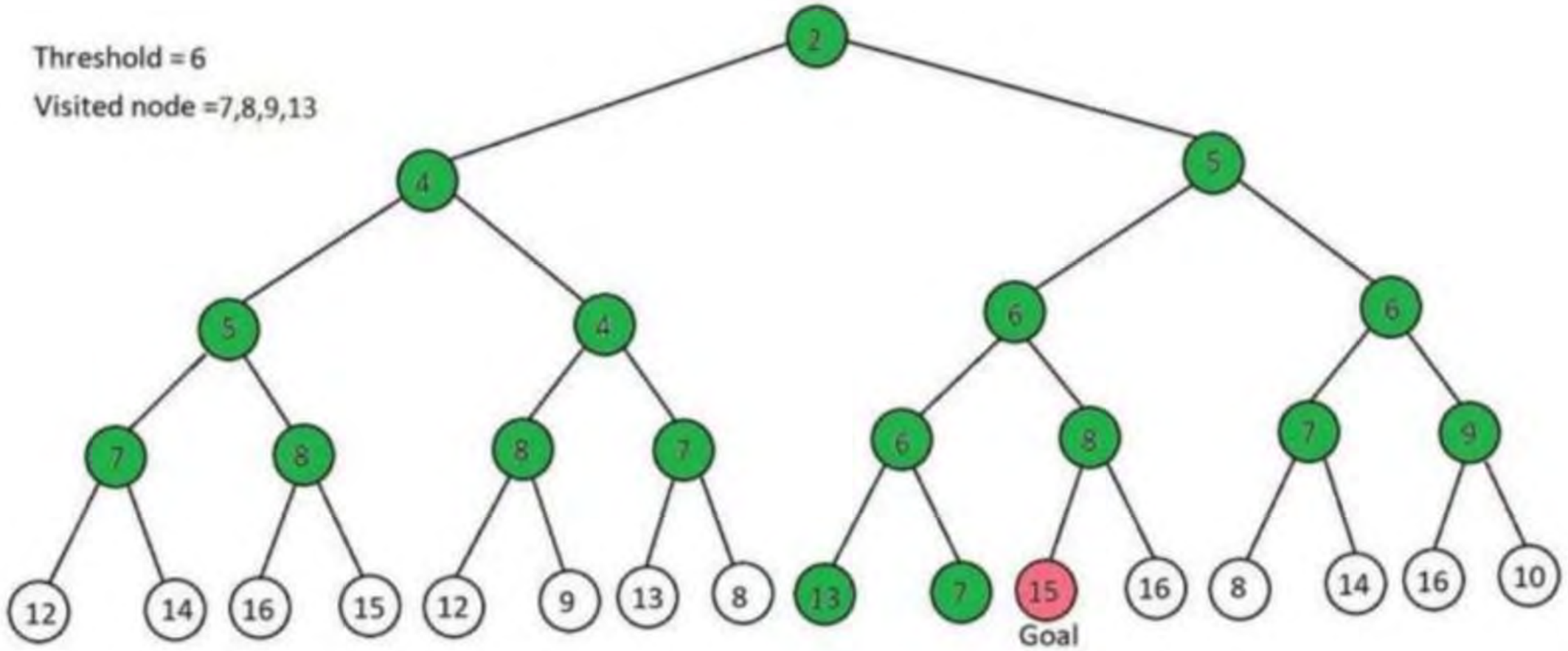

Threshold = 2
Visited node = 4, 5
Start
2
4
5
5
4
6
6
7
8
8
7
6
8
7
9
12
14
16
15
12
9
13
8
13
7
18
16
8
14
16
10
Goal


Threshold = 4
Visited node = 5,8,7
2
4
5
5
4
6
6
7
8
8
7
6
8
7
9
12
14
16
15
12
9
13
8
13
7
15
Goal
16
8
14
16
10


Threshold = 5
Visited node = 7,8,6
2
4
5
5
4
6
6
7
8
8
7
6
8
7
9
12
14
16
15
12
9
13
8
13
7
15
Goal
16
8
14
16
10

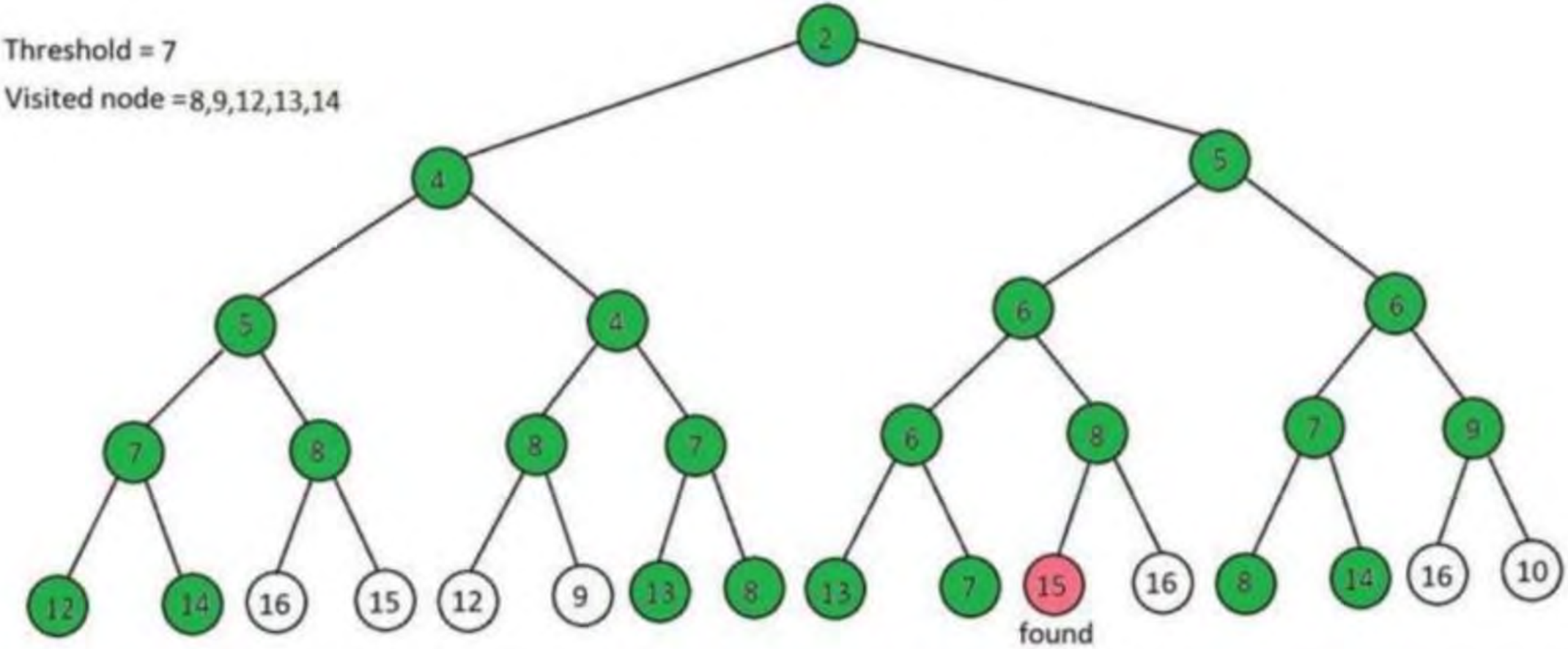
Threshold = 6

Visited node =7,8,9,13



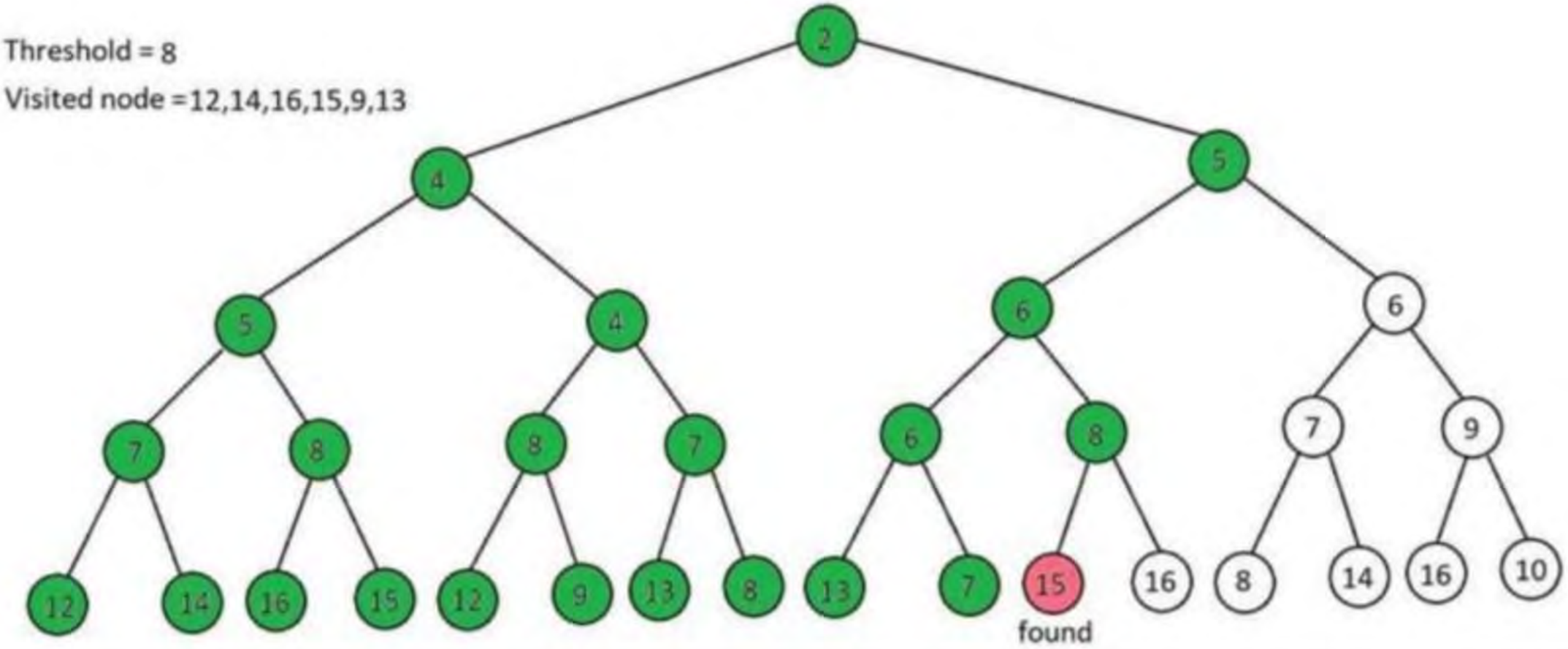
Threshold = 7

Visited node =8,9,12,13,14



Threshold = 8

Visited node =12,14,16,15,9,13



IDA* vs A*

- 1. Algorithm Type:
- A*: A* is a best-first search algorithm that explores nodes in the order of their estimated total path cost $f(n)=g(n)+h(n)$, where $g(n)$ is the cost to reach the node and $h(n)$ is the heuristic estimate of the cost from the node to the goal.
- IDA*: IDA* is an iterative deepening version of A*, which uses depth-first search but applies a threshold based on $f(n)$ to limit the depth of the search in each iteration. It increases the threshold progressively until the goal is found.

IDA* vs A*

- 2. Memory Usage:
- A*: Requires significant memory, as it stores all the nodes in the open and closed lists. In the worst case, this can lead to memory consumption growing exponentially with the depth of the search space.
- IDA*: Requires much less memory, similar to depth-first search. It only needs to store the current path and a small amount of additional information, making it much more memory-efficient.

IDA* vs A*

- 3. Time Complexity:
- A*: The time complexity is $O(b^d)$, where bb is the branching factor and dd is the depth of the optimal solution. It explores all possible paths but typically does so faster than uninformed search algorithms.
- IDA*: The time complexity is also $O(b^d)$, but due to its iterative nature, it may revisit nodes multiple times, making it potentially slower in practice compared to A*.

IDA* vs A*

- 4. Space Complexity:

- A*: Space complexity is $O(b^d)$, as it needs to store all explored nodes. This can be a significant limitation for large problems.
- IDA*: Space complexity is $O(b \times d)$, as it only stores the path being explored and the next threshold. This makes IDA* feasible for much larger problems than A*.

IDA* vs A*

- A* is faster in practice and is preferred when memory is sufficient.
- IDA* is more memory-efficient and suitable for large problems but may be slower due to its iterative nature.

THANK - YOU